

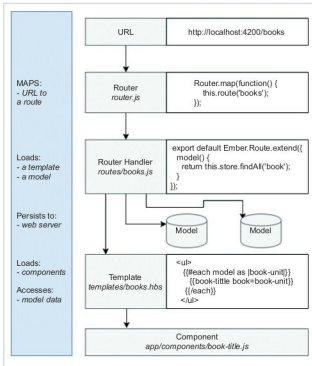


Figure 1: The core concepts of the Ember.js framework

does this by matching the current URL to the routes we've defined. Each URL has a corresponding route object that controls what users can see.

The Ember router maps the URL to a route handler, which renders a template and also loads a model, which is then available to the template.

Templates

Like the AngularJS app, the Ember app also uses templates for organising the layout of HTML. The templates in an Ember app look like any HTML fragment.

An example is:

```
<div>This is a valid template </div>
```

Ember templates use the syntax of Handlebar templates. The Handlebars syntax is used to build the DOM app. Templates can also display properties provided to them by the context.

Models

Models represent a persistent state. Every route has a model associated with it, which contains the data associated with it along with the current state of the app. Models can be configured to be saved somewhere else, like in the browser's local storage.

Components

Components control how the user interface works. Components consist of two parts—a template layout written in the Handlebars syntax, and a source file written in

JavaScript that defines the logic behind the controller.

Installing Node.js

To use Ember CLI to build our app, Node.js must be installed first. Download and install Node.js (<https://www.nodejs.org>). The Ember CLI is distributed as an npm package, so we will use Node and npm to install the Ember CLI.

Installing Ember

Open a terminal/command prompt and type the following command:

```
c:\>npm install -g ember-cli
```

This command will install EmberJS and all its dependencies.

Testing the installation

After installing the Ember CLI via npm, let's check if everything is installed properly. Once we have installed the Ember CLI, we will have access to the 'ember' command in the terminal/command prompt. Let's use the 'ember new' command to create a new sample application by running the following command:

```
c:\>ember new my-project
```

This will create a directory called 'my-project', and will set up a default new Ember application inside it. This application will create default configuration files and will also include the following:

- Development server
- Template compilation
- CSS and JavaScript minification
- The ES2015 feature via Babel

Now let us check if everything is working fine. Change the working directory to the 'my-project' directory by running the following command:

```
c:\>cd my-project
```

Now, start the development server by running the following command:

```
c:\my-project>ember server
```

After a few seconds, you will see the output in your terminal or command prompt, and it will look like what's shown below:

```
Livereload server on http://localhost:49152
Serving on http://localhost:4200/
```

Open <http://localhost:4200> in your browser. You should see an Ember welcome page, as shown in Figure 2.